Sherlock.ai: how it works

Oct 13, 2026 · @Raavan

The idea in one page

Sherlock.ai is an investigator for document collections: give it a pile of documents on one topic and it shows which pieces of information can be trusted, where each came from, and which documents are affected when something turns out to be wrong or out of date.

The problem. Large archives such as the NAA hold thousands of documents on the same subject. Facts get repeated, updated, corrected and sometimes copied wrongly from one document to the next. Today a person has to read everything to find out that document 12 still quotes a figure that document 87 corrected two years ago. AI tools make this worse, because they summarise confidently without saying where a claim came from or whether it is still current.

What Sherlock does about it.

- Breaks every document into individual claims (small, checkable statements).
- Links each claim to the source it came from and every document that repeats it.
- Spots when a newer document supersedes, contradicts or corrects an older claim, and when independent sources agree.
- Shows the evidence side by side and lets a person decide. Nothing is marked false without a human confirming it.
- Never alters an original document. Every status change goes into a separate, permanent audit log.

Who uses it. Information managers, records officers and archivists. Sherlock does not replace them. It does the searching and cross-checking across thousands of pages so they can spend their time on judgement.

How it answers the hackathon problem

The statement asks: *how might we build information management know-how into AI so that users can better recognise and manage bias, misinformation, and disinformation?* Sherlock answers it by making the AI do what an information manager does (check provenance, appraise sources, track versions, keep an audit trail) and by showing the results to the user in a form they can act on.

Part of the statement	What Sherlock does
Information management know-how	Provenance (where a claim came from), appraisal (how reliable a source has been), version history, authenticity (hashes) and an audit trail.
Built into AI	The AI model does the claim extraction and comparison, but every result must carry its evidence and a confidence level.
Recognise	Flags appear on claims, sources and documents, each with the quoted text from both sides.
Manage	The user can confirm, dismiss, supersede or correct a claim, and every linked document updates.
Misinformation	A claim that a later document corrects is traced to every document still repeating it.
Disinformation	A source that keeps feeding claims later contradicted is surfaced and down-weighted. Altered documents fail their hash check.
Bias	A source-diversity view warns when a topic leans on a single source, department or viewpoint.
AI-generated content	AI summaries ingested into the collection are checked against their own sources like any other document.

Bias is the lightest part of the build, so be upfront about that when presenting: Sherlock makes the sources behind a topic inspectable, it does not remove bias.

A worked example: the 101st document

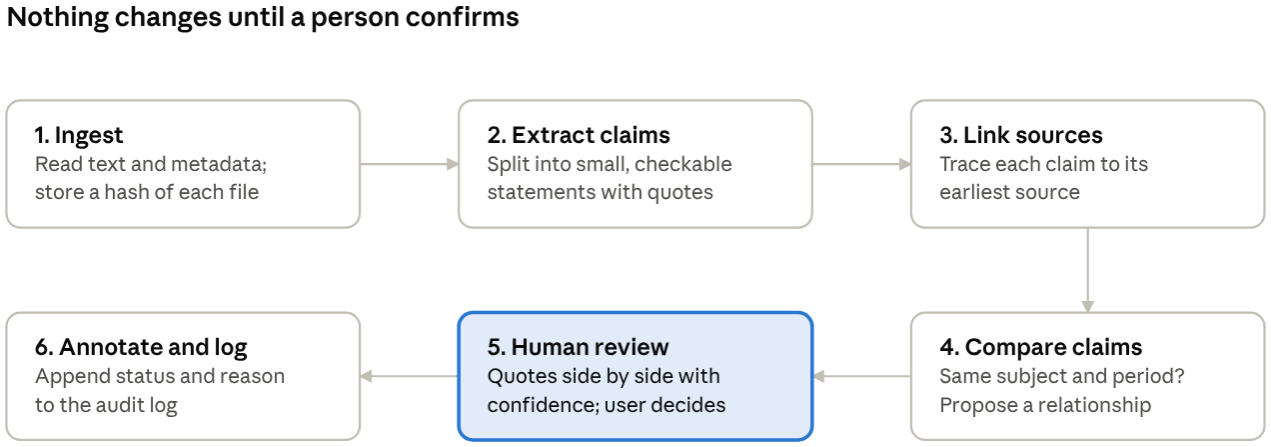
A records team uploads 100 documents about a department's annual leave policy. This is a made-up example to show the flow.

1. **Upload.** All 100 documents go in. Sherlock reads each one and extracts claims, for example "Annual leave is 20 days" or "Carry-over is capped at 10 days".
2. **Summary.** Sherlock shows the claims, the sources behind them and a reliability breakdown. "Annual leave is 20 days" appears in 31 documents. It traces back to two original sources, the signed HR policy and an older handbook. The other 29 documents repeat one of them.

3. **Click a source.** Opening the handbook shows the 14 documents that cite it, how many of its claims were later corroborated, superseded or contradicted, and when it was most active.
4. **The 101st document arrives.** It is a signed policy update stating annual leave is now 25 days, effective 1 July 2026.
5. **Sherlock investigates.** It finds 31 documents carrying the 20-day claim. It marks the claim as *superseded* (not false: it was correct until 1 July), lists the affected documents, and shows the old and new text side by side.
6. **The user decides.** They confirm the supersession for 29 documents and dismiss two where the 20-day figure applied to a different employee category. Both decisions go into the audit log with name, time and reason.
7. **Source reliability updates.** The handbook now has a visible record: one of its claims was superseded, which counts differently from being wrong.

The point of the example: one new document triggers a traceable, reviewable chain of consequences instead of a manual re-read of 100 files.

How it works, step by step



Sherlock pipeline · 6 steps, 1 human gate

Six steps take a document from upload to a recorded decision. Only step 5 can change what the system treats as settled, and it is always a person.

1. **Ingest.** Each document is read, its text and metadata extracted, and a hash of the original file stored. The file itself is kept unchanged.
2. **Extract claims.** The model splits each document into small, checkable statements (a figure, a date, a rule). Each claim keeps the exact quoted passage and its page, and gets its own hash.

3. **Link sources.** Every claim is traced to the earliest document or source that states it. Sherlock records which documents repeat it, so clicking a source lists every connected document.
4. **Compare claims.** For each new claim, Sherlock looks for existing claims about the same subject and time period and proposes how they relate: supersedes, contradicts, corrects or corroborates.
5. **Human review.** The reviewer sees both quotes side by side, with a confidence level, and confirms, changes or dismisses the proposal.
6. **Annotate and log.** The decision is appended to the audit log with who, when and why. Originals and hashes are never touched, and the reliability record of each affected source updates.

Four ways two claims can relate

A newer document is not automatically right, and an older one is not automatically wrong. Sherlock therefore never uses a single "true or false" switch. It labels how one claim relates to another, and a person confirms.

Relationship	Meaning	Example	What Sherlock does
Supersedes	Was correct then, replaced now	Leave policy v2 is replaced by v3	Marks the older claim as superseded and links every document that still uses it.
Contradicts	Two claims about the same thing conflict and neither is known to be right	Two reports give different budgets for the same year	Flags a conflict, shows both quotes, and asks the user to resolve it.
Corrects	A later document explicitly says an earlier one was wrong	An errata notice	Marks the earlier claim as corrected and traces everything that relied on it.
Corroborates	Independent sources agree	Two separate audits report the same figure	Raises confidence, but only if the sources are independent.

Two checks run before any relationship is proposed. **Same subject?** A claim about casual staff must not be compared with a claim about permanent staff. **Same time period?** A 2019 figure and a 2024 figure can both be right.

How source reliability is scored

Every data source gets a **reliability record**, not a single mystery number. We avoid the word "accuracy" because Sherlock has no ground truth to measure against. It can only measure how a source's claims have held up against everything else in the collection.

What the record shows

- How many claims the source provided.
- How many were corroborated by an independent source, superseded, corrected or contradicted.
- The period when the source was most active.
- The documents that cite it (click to open them).

A source might read: *47 claims: 41 corroborated, 4 superseded, 2 contradicted.* The score is calculated from those counts and the calculation is always visible.

Count independent sources, not documents. If 25 documents all copied the same wrong figure from one original, that is one source with 25 echoes, not 25 votes. Sherlock traces claims back to the earliest source and counts that once. Without this, repetition would look like proof. This is called circular reporting, and it is the most common way wrong information spreads through an archive.

Superseded is not the same as wrong. A source that was correct when written and later replaced is penalised far less than one whose claims were contradicted while still current.

Consensus is a signal, not proof. Even claims that every document agrees on are labelled *consistent across N independent sources*, never "verified true".

Keeping the originals untouched

Archives must preserve records exactly as they were created. Sherlock therefore works on two separate layers.

Layer	Contents	Can it change?
Record layer	The original documents, each with a hash (a fingerprint of its exact contents), plus a hash for every extracted claim.	Never. Any change breaks the hash and is flagged as tampering.
Annotation layer	Status and judgement: superseded, corrected, disputed, dismissed, who decided, when, and why.	Only by appending a new entry. Old entries are never edited or deleted.

So "marking a claim as false" never touches the claim or its hash. It adds a new annotation saying *"Claim C-0412 corrected by document D-101, confirmed by A. Smith on 14 Oct, reason: errata notice"*. The status shown on screen is the latest annotation, and the full history stays visible.

Why hashes. A hash is a short fixed-length code calculated from content. Change one character and the code changes completely. Storing the hash at the moment of upload lets Sherlock prove later that a document or claim is exactly what was ingested.

Why an append-only log. Each annotation includes the fingerprint of the previous one, so removing or editing an old entry breaks the chain and is detectable. This gives tamper evidence without needing a blockchain.

Privacy and deployment

Archives hold sensitive material, so Sherlock is designed to run entirely inside the organisation's own infrastructure. No document, claim or query is sent to an outside service.

- **Model.** An open-weight language model hosted on the organisation's own servers (for example through Ollama or vLLM). The model is a swappable component, so the organisation can use the strongest model its hardware supports.
- **Storage.** Documents, hashes, the claim graph and the audit log sit in the organisation's own database.
- **Access control.** Users see only the collections they are cleared for, and every view and decision is logged.
- **No training on their data.** Documents are used only for analysis in the user's session and are never used to train or fine-tune a model.

Honest trade-off. Models small enough to run on-site are usually less accurate than the largest cloud models. Sherlock compensates by design: every flag carries quoted evidence and a confidence level, and a human confirms before anything changes. A weaker model therefore produces more items for review, not more silent mistakes.

What the user sees and can do

Screen	What it shows	What the user can do
1. Upload	A drop zone for a batch of documents on one topic. Progress as each is read.	Add documents, name the collection, set access.

Screen	What it shows	What the user can do
2. Summary	The main claims, the sources behind them, a reliability breakdown, and a count of open flags.	Click any claim or source to drill down.
3. Claim view	One claim with every document that states it, the earliest source, and its status history.	Open linked documents, confirm or dismiss a suggested relationship.
4. Source view	One source with its reliability record, the documents that cite it, and when it was most active.	See which documents depend on a weak source.
5. Conflict review	Two claims side by side with the quoted passages highlighted, the proposed relationship and a confidence level.	Accept, reject, or change the relationship, and add a reason.
6. Impact view	After a new document or a decision: every document affected, grouped by how.	Review in bulk, export a list for correction.
7. Audit log	Every upload, flag and decision, with who, when and why.	Filter, export, verify the chain.
8. Diversity view	How many independent sources and departments sit behind each topic.	Spot single-source claims.

What each flag always includes: the exact quoted text, the document and page it came from, a confidence level, and the model version that produced it.

Safeguards, honest limits and judge questions

Limits we state openly

- Sherlock does not prove that anything is true. It shows how claims relate, how reliable sources have been, and what depends on what.
- The AI model can miss a contradiction or invent one. That is why every flag shows quoted evidence and waits for a human decision.
- Its judgements are only as good as the documents it is given. A topic with a single source stays marked as single-source.
- On-site models trade some accuracy for privacy.
- Bias is only partly addressed, through source diversity warnings.

Questions the judges are likely to ask

Question	Our answer
Are you replacing information managers?	No. Sherlock searches and cross-checks at a scale a person cannot, and the manager makes every decision.
Who decides what is false?	The user. Sherlock proposes a relationship with evidence and a confidence level, and nothing changes until it is confirmed.
What if the newest document is wrong?	Recency never decides. A newer claim creates a conflict or a supersession for review, not a verdict.
What if 25 documents just copy one source?	Claims are traced to the earliest source and counted once, so repetition is not treated as proof.
Can the AI hallucinate a contradiction?	Yes, so every flag shows both quotes side by side with a confidence level, and a person confirms it.
Does it change our records?	Never. Originals and their hashes stay untouched. Status lives in a separate, append-only audit log.
What about sensitive information?	Everything runs on the organisation's own servers with an open-weight model. Nothing leaves the building.
How does it handle bias?	It shows which sources, departments and viewpoints sit behind a topic and warns about single-source claims. It does not remove bias.

Demo script and what we need to build

Demo, about five minutes

1. Load a prepared collection of about 20 documents on one topic. A realistic small set demos better than 100 files.
2. Open the summary: claims, sources and reliability breakdown.
3. Click one source and show the documents linked to it and its history.
4. Upload a new document that supersedes one claim and contradicts another.

5. Show Sherlock's impact view: the affected documents, with quotes side by side.
6. Confirm one flag and dismiss another. Show both appear in the audit log.
7. Edit one original document on disk and re-run the check to show the hash mismatch.
8. Close on the diversity view: one topic resting on a single source.

Components to build

Component	What it does	Priority
Ingestion	Reads documents, extracts text and metadata, stores each with its hash.	Must have
Claim extraction	Uses the model to split each document into claims with the quoted passage and location.	Must have
Claim graph	Stores claims, sources, documents and the links between them.	Must have
Relationship engine	Proposes supersedes, contradicts, corrects or corroborates, with evidence and confidence.	Must have
Review interface	The screens listed above, with confirm and dismiss.	Must have
Audit log	Append-only, hash-chained record of every decision.	Must have
Reliability scoring	Counts per source, with independence handled.	Should have
Diversity view	Source and department spread per topic.	Should have
Role-based access	Collection-level permissions.	Nice to have